INACAP · Programación Back End (TI3041) · Proyecto transversal: Catálogo Online

Evaluación Sumativa 3 (ES3)

Unidad 2: Framework back end · ORM, reportes y formulario de clientes

1. Información general

Campo	Detalle
Evidencia	Aplicación web con CRUD, gestión de colecciones y manejo de sesiones, con apoyo de IA (proyecto publicado en GitHub)
Aprendizaje esperado	2.1 Codifica aplicaciones web para proponer soluciones tentativas a una problemática, utilizando un framework del lado del servidor y herramientas de inteligencia artificial como apoyo al desarrollo.
Criterios de evaluación	2.1.1 · 2.1.3 · 2.1.4
Ponderación	30% de la asignatura
Modalidad	Individual, presencial, en laboratorio
Duración	90 minutos
Instrumento	Escala de apreciación
Uso de IA	N2 (copiloto) con apoyo de IA para la codificación del sitio: CRUD, consultas del ORM, formularios y templates, adaptando lo generado. Toda consulta se registra en uso_ia.md.
Trazabilidad	Repositorio GitHub propio de esta evaluación, con un commit al cierre de cada etapa.
Punto de partida	Desde cero, desde su proyecto de la ES2 o desde el repositorio base del docente (con la BD poblada).

2. Proyecto transversal: Catálogo Online

Durante el semestre usted construye, por etapas, la plataforma web de un negocio local: primero la interfaz (ES1), luego la administración del contenido con base de datos (ES2), después reportes y formulario de clientes (ES3) y finalmente la API REST para integraciones (ES4).

En esta evaluación: el negocio ahora gestiona su catálogo directamente desde el sitio (CRUD completo desde la web), pide información (dos reportes con ORM) y contacto con clientes (formulario guardado en la BD), más una funcionalidad simple con sesiones.

Repositorio y punto de partida: cada evaluación se trabaja en un repositorio GitHub nuevo e independiente, creado al inicio de la prueba con el nombre apellido-ti3041-esN. Puede partir desde cero o desde el repositorio base publicado por el docente; partir desde cero incluye reutilizar el código de su propia evaluación anterior.

Personalización por sección: el docente indica la variante al inicio.

Sección	Negocio	Campos mínimos del dato principal	Volumen
A	Ferretería	nombre, categoría, precio, stock	40 productos
B	Minimarket	nombre, marca, precio, disponible	60 productos
C	Farmacia	nombre, laboratorio, precio, requiere_receta	30 productos

3. Estructura de la evaluación

El trabajo se organiza en etapas. Cada etapa termina con un commit en GitHub con el mensaje indicado. El registro de consultas a la IA se completa durante todo el desarrollo.

Etapa	Actividad	Commit
0. Entorno	Nuevo repositorio con el proyecto de la ES2 operativo y BD poblada	etapa-0-entorno
1. CRUD web	CRUD del catálogo desde la web: crear, editar y eliminar con reflejo en el listado	etapa-1-crud
2. Reportes ORM	Dos reportes construidos con consultas del ORM, con sus templates y navegación	etapa-2-reportes
3. Clientes y sesiones	Formulario de consultas con ModelForm, funcionalidad con sesiones y seguridad	etapa-3-clientes
4. Entrega	Documento de uso de IA, push final y verificación del repositorio	entrega-final

4. Etapa 0: Nuevo repositorio y punto de partida

- Cree el repositorio apellido-ti3041-es3 y elija su punto de partida: desde cero, su proyecto de la ES2 o el repositorio base del docente. Instale dependencias, migre y cargue el poblamiento; verifique que el listado se muestra desde la BD.
- Commit etapa-0-entorno.

5. Etapa 1: CRUD del catálogo desde la web

- Implemente las operaciones desde el sitio (no solo desde /admin): crear un registro con formulario, editar un registro existente y eliminar con confirmación.
- Tras cada operación, redirija al listado principal para ver el cambio reflejado desde la BD.
- Puede apoyarse en IA para las vistas y formularios del CRUD, adaptando lo generado a su modelo. Commit etapa-1-crud.

6. Etapa 2: Reportes con ORM

- **Reporte 1, resumen por agrupación:** cantidad de registros y algún cálculo (promedio de precio, total de stock) por categoría, marca o laboratorio, usando aggregate o annotate del ORM.
- **Reporte 2, listado filtrado:** registros bajo un umbral (stock crítico, no disponibles, o que requieren receta), usando filter y order_by.
- Cada reporte tiene su template, ambos accesibles desde la navegación de base.html, con los cálculos resueltos en la vista con el ORM. Commit etapa-2-reportes.

7. Etapa 3: Formulario de clientes, sesiones y seguridad

- Cree el modelo ConsultaCliente (nombre, correo, mensaje, producto de interés opcional, fecha automática), registrado en /admin.
- Implemente el formulario con ModelForm, con validaciones (campos obligatorios, correo válido, largo mínimo); la vista guarda en la BD y muestra confirmación.
- Implemente una funcionalidad simple con sesiones (últimos 3 productos visitados o contador de visitas) y verifique CSRF y validaciones con datos inválidos.
- Commit etapa-3-clientes.

8. Documento de uso de IA (uso_ia.md)

Cree en la raíz del repositorio el archivo uso_ia.md con dos partes:

Parte 1, registro de consultas: cada consulta realizada a la IA durante la evaluación: el prompt textual, un resumen de la respuesta y qué usó o modificó antes de integrarla.

Parte 2, explicación del proceso: un texto de 10 a 20 líneas donde explica CON SUS PROPIAS PALABRAS cómo usó la IA: qué le pidió y por qué, qué respuestas le sirvieron tal cual, cuáles tuvo que corregir y qué aprendió en el proceso.

No se aceptan en la Parte 2 textos redactados o resumidos por IA: la redacción debe ser personal y se contrasta en la revisión con el docente. Un texto genérico que no se condice con el trabajo del repositorio se considera no logrado.

9. Revisión personal con el docente

Al cierre de la evaluación (o durante la clase siguiente, según lo organice el docente), cada estudiante tiene una revisión individual de 5 minutos:

- Muestra su aplicación funcionando desde el repositorio entregado.
- El docente elige un fragmento del código al azar y el estudiante lo explica.
- Se contrasta uso_ia.md con el código: las consultas declaradas deben corresponder al trabajo realizado.

Esta revisión respalda los indicadores de comprensión de la escala de apreciación. No poder explicar el propio código afecta directamente esos indicadores.

10. Escala de apreciación

Niveles: L = Logrado (2 pts) · ML = Medianamente logrado (1 pt) · NL = No logrado (0 pts). Puntaje máximo: 26 pts. Exigencia 60%.

N	Indicador	Criterio	Pts
1	Nuevo repositorio operativo con la BD poblada y el listado funcionando	2.1.1	
2	Crear y editar registros desde la web, con reflejo en el listado	2.1.3	
3	Eliminar con confirmación desde la web, con reflejo en el listado	2.1.3	
4	Reporte 1 con agregación del ORM (aggregate o annotate) correcto	2.1.3	
5	Reporte 2 con filtro y orden del ORM correcto, ambos navegables desde base.html	2.1.3	
6	Cálculos resueltos en la vista con el ORM, no en el template	2.1.3	
7	Modelo de consultas creado, migrado y registrado en /admin	2.1.1	
8	ModelForm con validaciones operativas; guarda en BD y confirma al cliente	2.1.4	
9	Funcionalidad de sesiones operativa y token CSRF presente	2.1.4	
10	Historial GitHub con los 5 commits de etapa y progresión coherente	Trazabilidad	
11	uso_ia.md Parte 1 completa: consultas con prompt, resumen y adaptación	Uso de IA	
12	uso_ia.md Parte 2 con redacción personal, coherente con el repositorio	Uso de IA	
13	Revisión personal: ejecuta su aplicación y explica el fragmento consultado	Comprensión	

Anexo (docente)

- Probar el formulario con datos inválidos (correo malo, campos vacíos): debe rechazar con mensajes claros.
- Verificar los reportes contra la BD (los totales deben cuadrar con /admin).
- Probar el ciclo CRUD completo desde la web: crear, editar y eliminar un registro, verificando el reflejo en el listado.